Retailmind-Autonomous Retail Research Agent

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Arsh Patidar	**EN22CS301204**
Aniket Kushwah	**EN22CS301124**
Avani Gupta	**EN22CS301236**
Anuj Singh Rathore	**EN22CS301166**
Amit Patidar	**EN22CS301114**
Anurag Didolkar	**EN22CS301169**

Under the Guidance of

Prof. Akshay Saxena

Dr. Hemlata Patel

Prof. Ajeet Singh Rajput

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Retailmind - Autonomous Retail Research Agent

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Arsh Patidar EN22CS301204
Aniket Kushwah EN22CS301124
Avani Gupta EN22CS301236
Anuj Singh Rathore EN22CS301166
Amit Patidar EN22CS301114
Anurag Didolkar EN22CS301169

Under the Guidance of

Prof. Akshay Saxena
Dr. Hemlata Patel
Prof. Ajeet Singh Rajput

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

REPORT APPROVAL

The project work **"Retailmind – Autonomous Retail Research Agent"** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.
It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

External Examiner Name

Name:

DECLARATION

We hereby declare that the project entitled **"Retailmind – Autonomous Retail Research Agent "** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and engineering completed under the supervision of **Prof. Akshay Saxena, Dr. Hemlata Patel, Prof. Ajeet Singh Rajput,** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Arsh Patidar - EN22CS301204

Aniket Kushwah - EN22CS301124

Avani Gupta - EN22CS301236

Anuj Singh Rathore - EN22CS301166

Amit Patidar - EN22CS301114

Anurag Didolkar - EN22CS301169

CERTIFICATE

We, **Prof. Akshay Saxena, Dr. Hemlata Patel, Prof. Ajeet Singh Rajput** certify that the project entitled "**Retailmind – Autonomous Retail Research Agent "** submitted in partial fulfillment for the award of the degree of Bachelor
of Technology/Master of Computer Applications by **Arsh Patidar, Aniket Kushwah, Avani Gupta, Anuj Singh Rathore, Amit Patidar, Anurag Didolkar** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Hemlata Patel
Computer Science & Engineering

Prof. Ajeet Singh Rajput
Computer Science & Engineering

Prof. Akshay Saxena
Computer Science & Engineering

Mr. Suraj Nayak

Mr. Ashwin Krishnan CK

Mr. Vaibhav

Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medicaps University, Indore

=

ACKNOWLEDMENT

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express our heartfelt gratitude to our Internal Guide, **Prof. Akshay Saxena**, **Dr. Hemlata Patel, Prof. Ajeet Singh Rajput** without whose continuous help and support, this project would ever have reached to the completion.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Arsh Patidar
Aniket Kushwah
Avani Gupta
Anuj Singh Rathore
Amit Patidar
Anurag Didolkar

B.Tech. IV Year
Department of Computer Science &
Engineering Faculty of Engineering
Medicaps University, Indore

ABSTRACT

RetailMind – Autonomous Retail Research Agent is a full-stack AI-powered application that enables users to conduct deep retail industry research by simply entering a natural language query. The system employs three autonomous AI agents — a Researcher, an Analyst, and a Writer — built using CrewAI and powered by Google Gemini 2.0 Flash, to search the web via Tavily Search API, extract structured insights, and generate a comprehensive professional research report.

The backend is developed using Python 3.11 and FastAPI, while the frontend is built with React 18 and Vite, styled using Tailwind CSS. Research progress is streamed live to the browser via Server-Sent Events (SSE), allowing users to watch each agent work in real time without any page refresh. Every generated report is saved in dual storage: a ChromaDB vector database for semantic search and a plain-text (.txt) file for human-readable backup.

The entire application is containerized using Docker with multi-stage builds and deployed to an AWS EC2 instance. A complete DevOps pipeline is implemented using GitHub Actions — including a CI workflow that runs on every push and pull request, a CD workflow that builds, pushes, and deploys Docker images to EC2 on every merge to main, and a Rollback workflow that allows one-click rollback to any previous deployment using git SHA-tagged Docker images.

Keywords:
Generative AI, Multi-Agent Systems, CrewAI, LangChain, Google Gemini 2.0, FastAPI, React, ChromaDB, Tavily Search, Server-Sent Events, Docker, GitHub Actions, AWS EC2, CI/CD, Vector Database.

[1]

Table of Contents

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CrewAI	Crew Artificial Intelligence — multi-agent orchestration framework
EC2	Elastic Compute Cloud
FastAPI	Fast Application Programming Interface framework for Python
LLM	Large Language Model
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SHA	Secure Hash Algorithm — used for versioned Docker image tags
SPA	Single Page Application
SSE	Server-Sent Events
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Introduction

RetailMind – Autonomous Retail Research Agent is a full-stack, AI-powered web application that automates retail industry research. The user submits a single natural language query and the system autonomously produces a comprehensive, well-structured report by orchestrating three specialised AI agents: a Researcher who gathers raw information from the web, an Analyst who synthesises it into structured insights, and a Writer who produces the final professional report — all without any human intervention.

The backend is built on Python 3.11 and FastAPI, with a CrewAI multi-agent pipeline powered by Google Gemini 2.0 Flash and Tavily Search API for real-time web search. The frontend is a React 18 SPA built with Vite and Tailwind CSS. Research progress is streamed live to the browser via Server-Sent Events (SSE). Every generated report is saved in ChromaDB for semantic search and as a plain-text file for backup.

The entire application is containerised using Docker, orchestrated with Docker Compose, and deployed to AWS EC2. A GitHub Actions CI/CD pipeline automates testing, building, and deployment on every commit. SHA-versioned Docker image tags enable one-click rollback to any previous deployment.

1.2 Literature Review

The development of autonomous research agents has been driven by advances in Large Language Models and multi-agent orchestration frameworks. Research on role-based agent systems has shown that assigning specialised roles to individual agents — rather than using a single general agent — significantly improves task quality and reduces hallucinations. Each agent in RetailMind has a clearly defined role, goal, backstory, and constrained tool access, following established best practices for multi-agent design.

The Tavily Search API, used by the Researcher agent, is specifically designed for LLM consumption, returning clean structured snippets rather than raw HTML. This approach of grounding LLM outputs in retrieved real-time content — known as Retrieval-Augmented Generation (RAG) — has been demonstrated to significantly improve factual accuracy. ChromaDB with all-MiniLM-L6-v2 embeddings enables semantic similarity search, allowing retrieval based on meaning rather than keywords.

In the DevOps domain, containerisation with Docker, CI/CD with GitHub Actions, and SHA-based versioned deployments are well-established industry practices that this project applies to an AI application. The SSE streaming pattern has been shown to improve user trust and engagement for AI systems with longer processing times by making the process transparent and observable.

1.3 Objectives

The specific objectives of the RetailMind project are as follows:

Design and implement a three-agent CrewAI pipeline (Researcher, Analyst, Writer) that produces comprehensive retail research reports from a single natural language query.
Integrate Google Gemini 2.0 Flash as the LLM backbone and Tavily Search API for real-time web research.
Implement Server-Sent Events (SSE) streaming to broadcast live agent progress to the frontend in real time.
Build dual-storage persistence: ChromaDB vector database for semantic search and plain-text files for human-readable backup.
Develop a responsive React 18 frontend with research input, live pipeline visualisation, report viewing, and knowledge base search.
Containerise all components using Docker with multi-stage builds.
Implement a three-job GitHub Actions CD pipeline (test → build → deploy) with SHA-versioned images and one-click rollback.
Deploy the complete application on AWS EC2 with zero-downtime updates.

1.4 Significance

At the commercial level, RetailMind democratises retail market research. Generating professional research traditionally requires hours of manual effort or expensive third-party reports. RetailMind delivers the same quality in minutes at minimal cost — making competitive intelligence accessible to startups, students, and analysts alike.

At the technical level, the project demonstrates the integration of multi-agent AI, real-time streaming, vector database knowledge management, and production DevOps in a single cohesive system. The SHA-versioned deployment with automated rollback shows that AI applications can be built and maintained with the same rigour as enterprise production software.

At the educational level, the project provides end-to-end, hands-on experience across Generative AI, Agentic AI, and DevOps — the full AI application engineering lifecycle.

1.5 Research Design

The project follows a three-phase progressive methodology aligned with the Datagami Skill-Based Course Program:

Phase 1 — Generative AI (GAI-15): Designed the three-agent CrewAI pipeline, integrated Gemini 2.0 Flash and Tavily Search API, and built the initial FastAPI backend.
Phase 2 — Agentic AI (AAI-15): Added SSE streaming, asynchronous session management, ChromaDB knowledge base, and the React frontend.

Phase 3 — DevOps (DO-15): Wrote multi-stage Dockerfiles, designed GitHub Actions CI/CD workflows, implemented SHA-versioned deployments with rollback, and deployed to AWS EC2.

1.6 Source of Data

RetailMind uses the following data sources:

User natural language queries submitted through the React frontend.
Real-time web search results from Tavily Search API (sourcing McKinsey, Deloitte, Forbes, Retail Dive, NRF).
ChromaDB vector database (chroma_data Docker volume) for semantic search over generated reports.
Plain-text report files (kb_data Docker volume) for human-readable backup.
GitHub repository (awwniket47/Retailmind-Autonomous_Retail_Research_Agent) for version control and CI/CD.
AWS EC2 instance (Ubuntu 22.04 LTS) as cloud production infrastructure.

1.7 Chapter Scheme

Chapter 1: Introduction — Background, objectives, significance, and research design.
Chapter 2: Report on Present Investigation — Experimental setup, techniques, architecture overview, and performance metrics.
Chapter 3: Requirements Specification — Functional, non-functional, hardware, and software requirements.
Chapter 4: Proposed Solution and System Architecture — AI pipeline, backend, frontend, and DevOps architecture.
Chapter 5: Designs — Algorithms, DFDs, flowcharts, activity diagrams, class diagrams, and database design.
Chapter 6: Implementation, Testing and Maintenance — Key implementation details, test cases, and installation guide.
Chapter 7: Results and Discussions — Screenshots and discussion of outcomes.
Chapter 8: Summary and Conclusions — Summary of work done and key conclusions.
Chapter 9: Future Scope — Planned enhancements across AI, features, and DevOps.
Chapter 10: Appendix — API reference, environment variables, and file structure.
Chapter 11: Bibliography — All referenced books, papers, and online resources.

CHAPTER 2
REPORT ON PRESENT INVESTIGATION

2.1 Experimental Set-up

2.1.1 Backend Development Environment

The backend is a Python 3.11 FastAPI application running via Uvicorn at localhost:8000. It integrates Google Gemini 2.0 Flash via langchain-google-genai, Tavily Search via tavily-python, CrewAI 0.28.0 for multi-agent orchestration, and ChromaDB 0.5.3 for vector storage. Eight REST/SSE API endpoints are exposed, including POST /api/research (start session) and GET /api/research/{id}/stream (SSE stream).

2.1.2 Frontend Development Environment

The frontend is a React 18 SPA on Vite 5 at localhost:5173, styled with Tailwind CSS. Axios handles REST calls and the browser-native EventSource API consumes SSE streams. Three pages serve distinct functions: Research Page (query + live pipeline), Knowledge Page (semantic and keyword search), and History Page (past session browser).

2.1.3 Production Cloud Infrastructure

Production runs on an AWS EC2 Ubuntu 22.04 instance with Docker and Docker Compose. Two containers run in production: retailmind-backend (port 8000) and retailmind-frontend (port 80). Two persistent Docker volumes — kb_data and chroma_data — preserve research data across container restarts and redeployments.

2.2 Procedures Adopted

For the AI pipeline: three CrewAI agent roles were defined with tailored goals, backstories, and tool access; Gemini 2.0 Flash was integrated as the shared LLM; Task objects with detailed descriptions and expected output formats were assembled into a Crew with sequential execution; and the pipeline was wrapped in an asynchronous session manager.

For the DevOps pipeline: multi-stage Dockerfiles were written for both services; a three-job GitHub Actions CD workflow (test → build → deploy) was implemented; Docker Compose with health-check-driven startup was configured; and the rollback workflow was implemented using SHA-tagged image versioning.

2.3 Techniques Developed and Adopted

2.3.1 Three-Agent CrewAI Research Pipeline

The pipeline uses CrewAI with sequential process execution: Researcher (max_iter=6, TavilySearchTool) → Analyst (max_iter=4) → Writer (max_iter=3, no tools). Each agent focuses

exclusively on its assigned cognitive task. The Researcher performs up to six differently-worded search iterations to maximise coverage. The Analyst synthesises collected data into structured findings. The Writer produces the final markdown report from the Analyst's output.

2.3.2 Asynchronous Session Management with SSE Streaming

Sessions are managed via a shared Python dictionary keyed by UUID session IDs. The CrewAI pipeline runs in a ThreadPoolExecutor to avoid blocking the FastAPI event loop. An async SSE generator polls session state at 0.5-second intervals and yields formatted Server-Sent Events to the browser client, enabling real-time progress without WebSockets.

2.3.3 Dual-Storage Knowledge Base

Every completed report is written to two stores simultaneously: ChromaDB (dense vector embedding via ONNX all-MiniLM-L6-v2) for semantic similarity search, and a plain-text .txt file (with metadata header) for human-readable backup. This ensures reports are both semantically retrievable and directly accessible.

2.3.4 Docker Multi-Stage Build and SHA-Versioned Deployments

Multi-stage Dockerfiles reduce image sizes from ~400MB to ~25MB (frontend) and ~350MB (backend) by discarding build-stage dependencies. Every CD deployment builds and pushes Docker images with two tags: :latest and :<git-sha>. The deployed SHA is recorded in deployments.log on EC2, enabling the rollback workflow to revert to any previous version by SHA.

2.4 Data Collection and Preparation

Research data is collected at runtime: the user's query is passed to the Researcher agent, which retrieves real-time web snippets via Tavily. No pre-collected dataset is required. Completed reports are automatically persisted to ChromaDB and the plain-text file store. All infrastructure credentials (API keys, SSH keys, Docker Hub tokens) are stored as GitHub Secrets and injected at runtime — never committed to code or baked into images.

2.5 System Architecture Overview

2.5.1 Two-Tier Application Architecture

Tier 1 — Presentation: React 18 SPA (Nginx-served) providing research input, live SSE pipeline view, report rendering, and knowledge base search.

Tier 2 — Business Logic / AI: FastAPI backend hosting the CrewAI pipeline, session management, SSE streaming, and knowledge base operations.

Data Layer: ChromaDB (semantic search), plain-text file store (backup), in-memory dict (active sessions).

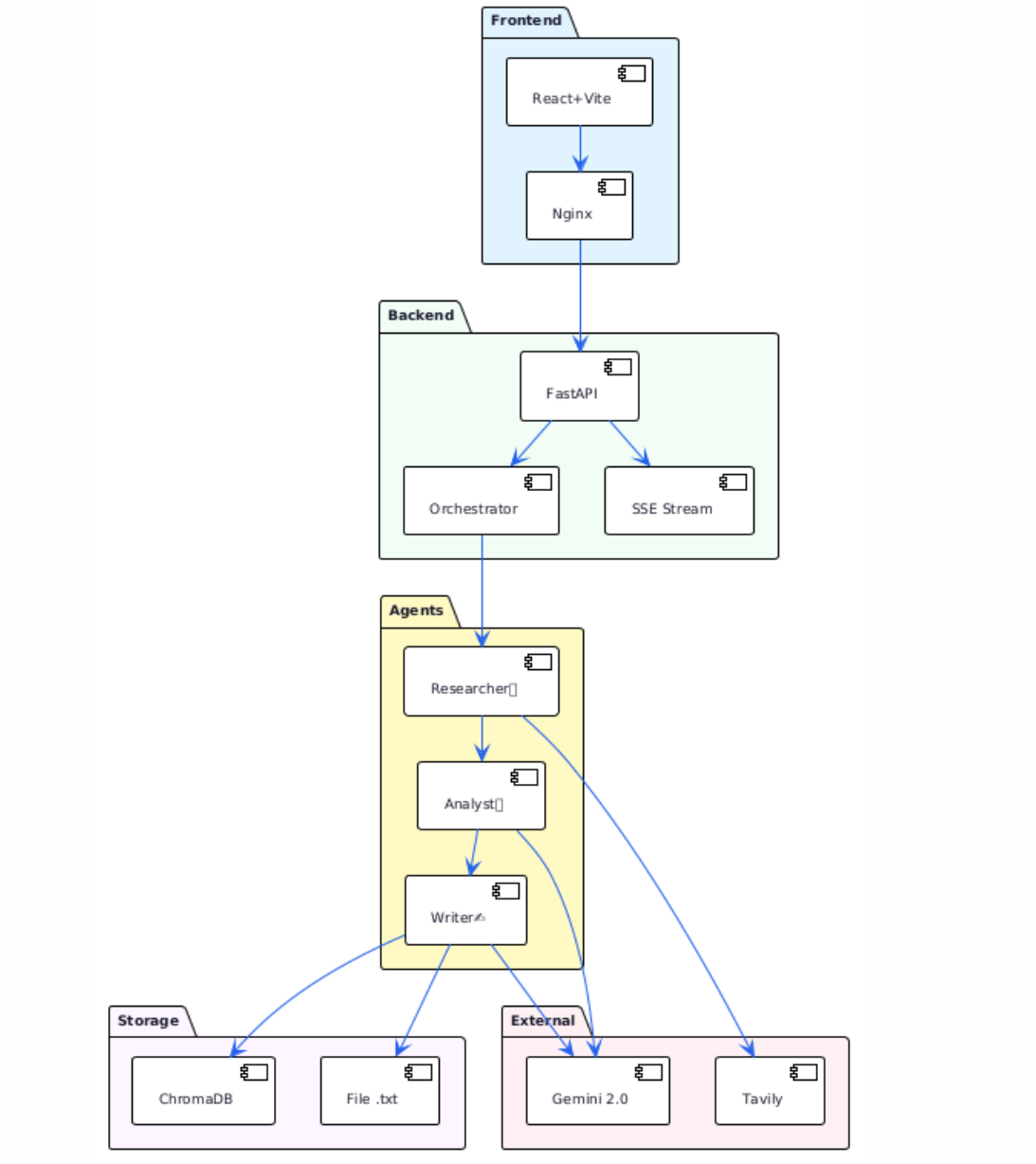

Figure 2.5.1: RetailMind System Architecture Overview

2.5.2 Three-Agent Pipeline Architecture

Agent 1 — Senior Retail Industry Researcher: Up to 6 Tavily search iterations, collecting raw retail industry information.

Agent 2 — Retail Market Intelligence Analyst: Synthesises collected data into key trends, statistics, and structured findings.

Agent 3 — Retail Research Report Writer: Produces the final professionally formatted markdown report.

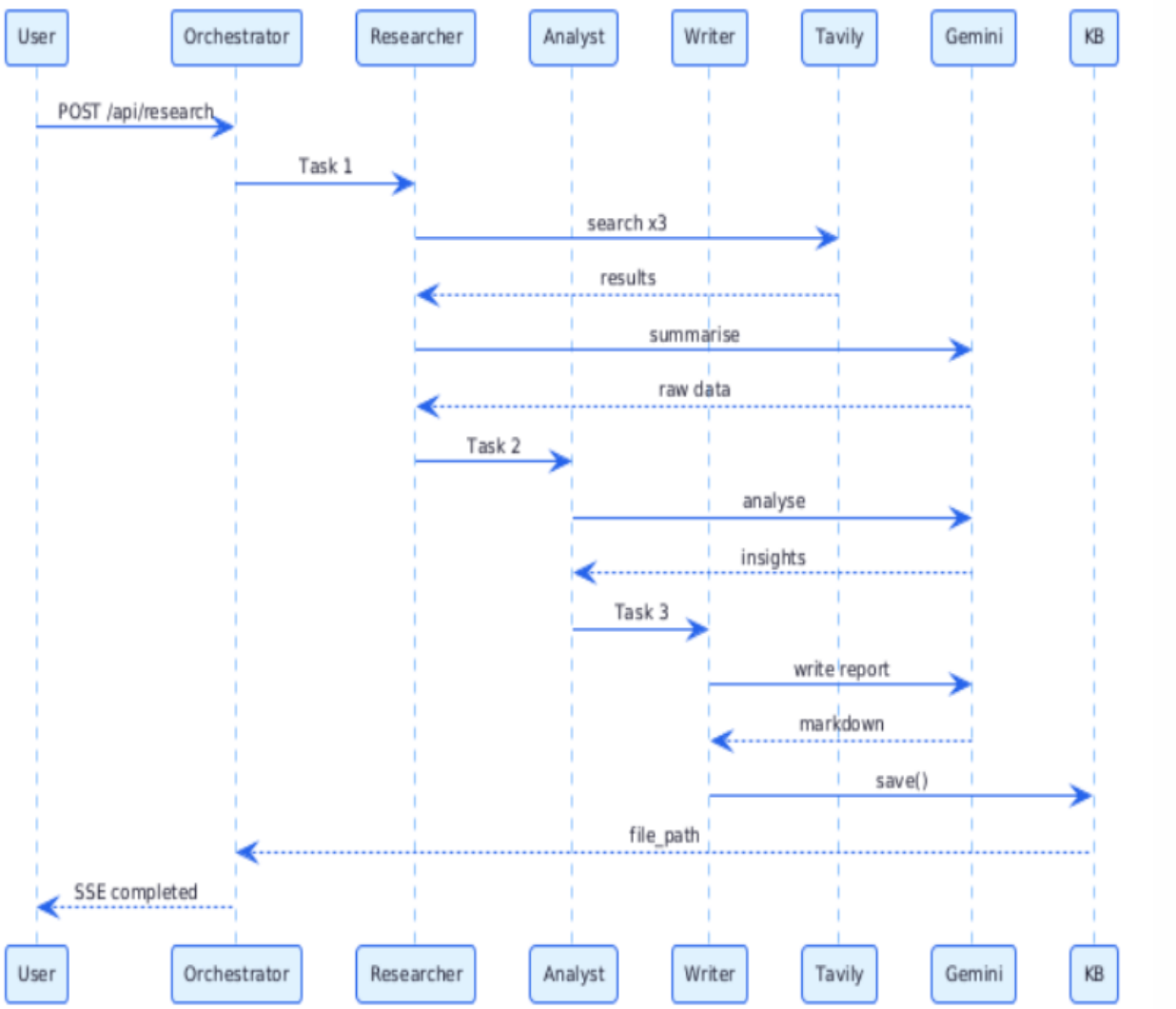

Figure 2.5.2: Three-Agent CrewAI Sequential Pipeline

2.5.3 CI/CD Pipeline Architecture

Job 1 — Test: Ruff lint, pytest backend tests, frontend build check.

Job 2 — Build and Push: Docker Buildx build, push :latest and :<git-sha> tags to Docker Hub.

Job 3 — Deploy: SSH to EC2, docker compose pull, docker compose up -d, record SHA in deployments.log.

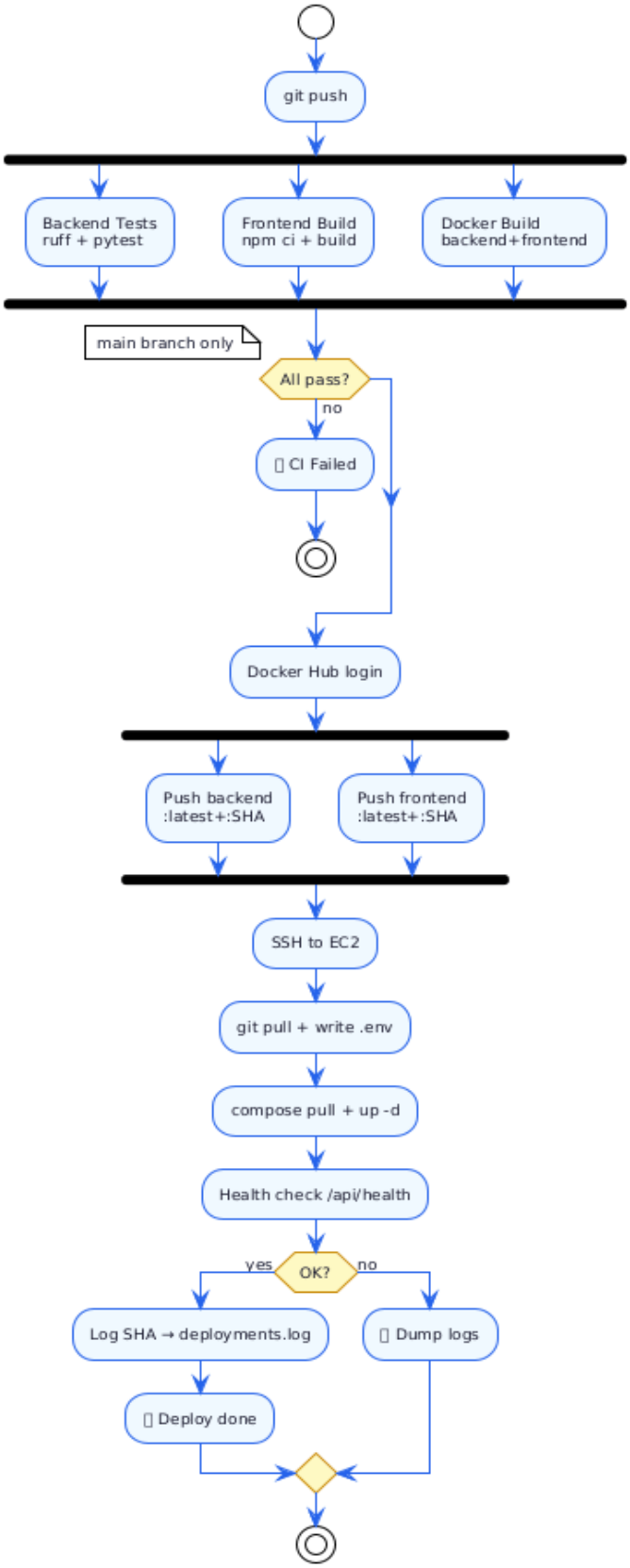

Figure 2.5.3: GitHub Actions CI/CD Pipeline Workflow

2.6 Technology Stack Summary

Category	Technology	Version	Purpose
Backend	FastAPI + Uvicorn	0.115.0 / 0.30.6	REST API and SSE streaming
Language	Python	3.11	Backend runtime
Multi-Agent	CrewAI	0.28.0	Three-agent pipeline orchestration
LLM	Google Gemini 2.0 Flash (via LangChain)	Latest	Content generation for all agents
Search	Tavily Python	0.3.3	Real-time web search for Researcher
Vector DB	ChromaDB	0.5.3	Semantic search for knowledge base
Validation	Pydantic	2.9.2	Request/response schema validation
Frontend	React 18 + Vite 5	-	Single-page application
Styling	Tailwind CSS	3.x	Utility-first styling
Containers	Docker + Docker Compose	Latest	Multi-container packaging
CI/CD	GitHub Actions	Latest	Automated test, build, deploy
Registry	Docker Hub	-	Image storage and SHA versioning
Cloud	AWS EC2 Ubuntu 22.04	t3.small	Production hosting
Web Server	Nginx Alpine	Latest	Static file serving and API proxy

Table 2.6: RetailMind Technology Stack

2.7 Performance Metrics and Evaluation

Metric	Target	Achieved
Research Report Generation	< 3 minutes	60 – 180 seconds
SSE Stream Latency	< 2 seconds/update	< 1 second
API Response (POST /research)	< 500 ms	< 200 ms
Frontend Build Size	< 5 MB	~2.5 MB
Backend Docker Image	< 500 MB	~350 MB
Frontend Docker Image	< 50 MB	~25 MB
CI Pipeline Duration	< 5 minutes	~3 minutes
CD Pipeline Duration	< 10 minutes	~5–6 minutes
Health Check Response	< 200 ms	< 100 ms
Zero-Downtime Deployment	Required	Achieved

Table 2.7: Performance Metrics

CHAPTER 3
REQUIREMENTS SPECIFICATION

3.1 User Characteristics

Retail Professionals: Brand managers and analysts requiring quick research without technical expertise.
Consultants and Investors: Users generating research briefs and evaluating market opportunities.
Students and Researchers: Users studying retail economics or consumer behaviour.
DevOps Engineers: Technical users maintaining and deploying the platform via GitHub Actions and AWS EC2.

3.2 Functional Requirements

FR ID	Requirement	Priority
FR-01	Accept a natural language query (5–300 chars) and start a research session.	High
FR-02	Execute a three-agent CrewAI pipeline (Researcher, Analyst, Writer) sequentially.	High
FR-03	Researcher agent performs up to 6 Tavily web search iterations.	High
FR-04	Stream live agent progress to the frontend via SSE without page refresh.	High
FR-05	Save every completed report to ChromaDB for semantic search.	High
FR-06	Save every completed report as a plain-text file with metadata header.	High
FR-07	Expose a semantic search endpoint over the knowledge base.	Medium
FR-08	Expose a keyword text search endpoint over the knowledge base.	Medium
FR-09	Display a live agent pipeline visualisation on the frontend.	High
FR-10	Render the completed report in formatted markdown on the frontend.	High
FR-11	CI pipeline runs backend tests and frontend build on every push.	High
FR-12	CD pipeline auto-deploys to EC2 on every merge to main.	High
FR-13	Rollback workflow enables one-click reversion to any SHA-tagged deployment.	High

Table 3.2: Functional Requirements

3.3 Non-Functional Requirements

NFR ID	Requirement	Category
NFR-01	Research report generation completes within 3 minutes.	Performance
NFR-02	Backend API responds to non-streaming requests within 500 ms.	Performance
NFR-03	CD pipeline completes end-to-end deployment within 10 minutes.	Performance
NFR-04	System supports at least 5 concurrent research sessions.	Scalability
NFR-05	All credentials stored as environment variables; none in source code.	Security
NFR-06	Zero-downtime deployments via docker compose pull + up -d.	Availability
NFR-07	Research data persists across container restarts via Docker volumes.	Reliability
NFR-08	All automated tests must pass before any deployment is triggered.	Quality
NFR-09	Docker images use multi-stage builds to minimise sizes.	Efficiency

Table 3.3: Non-Functional Requirements

3.4 Software Requirements

Component	Version	Purpose
Python	3.11+	Backend runtime
Node.js	20 LTS	Frontend build
Docker + Compose	24+ / v2.x	Containerisation
Google Gemini API Key	Active (AI Studio)	LLM access
Tavily API Key	Active (tavily.com)	Web search
Docker Hub Account	Free tier	Image registry
AWS Account	EC2 access	Cloud hosting
GitHub Account	Actions enabled	CI/CD and version control

Table 3.5: Software Requirements

CHAPTER 4
PROPOSED SOLUTION AND SYSTEM ARCHITECTURE

4.1 Solution Overview

RetailMind solves the retail research problem through three integrated layers: the AI pipeline (how research is generated), the application architecture (how the system is structured), and the DevOps infrastructure (how it is built and deployed).

The CrewAI multi-agent approach was chosen over a single-agent design because role specialisation improves output quality — each agent's prompt, backstory, and tool access are tuned for its specific task. SSE was chosen over polling or WebSockets for its native browser support and low infrastructure overhead. Docker Compose on a single EC2 instance was chosen over Kubernetes for its simplicity and cost-effectiveness at portfolio scale.

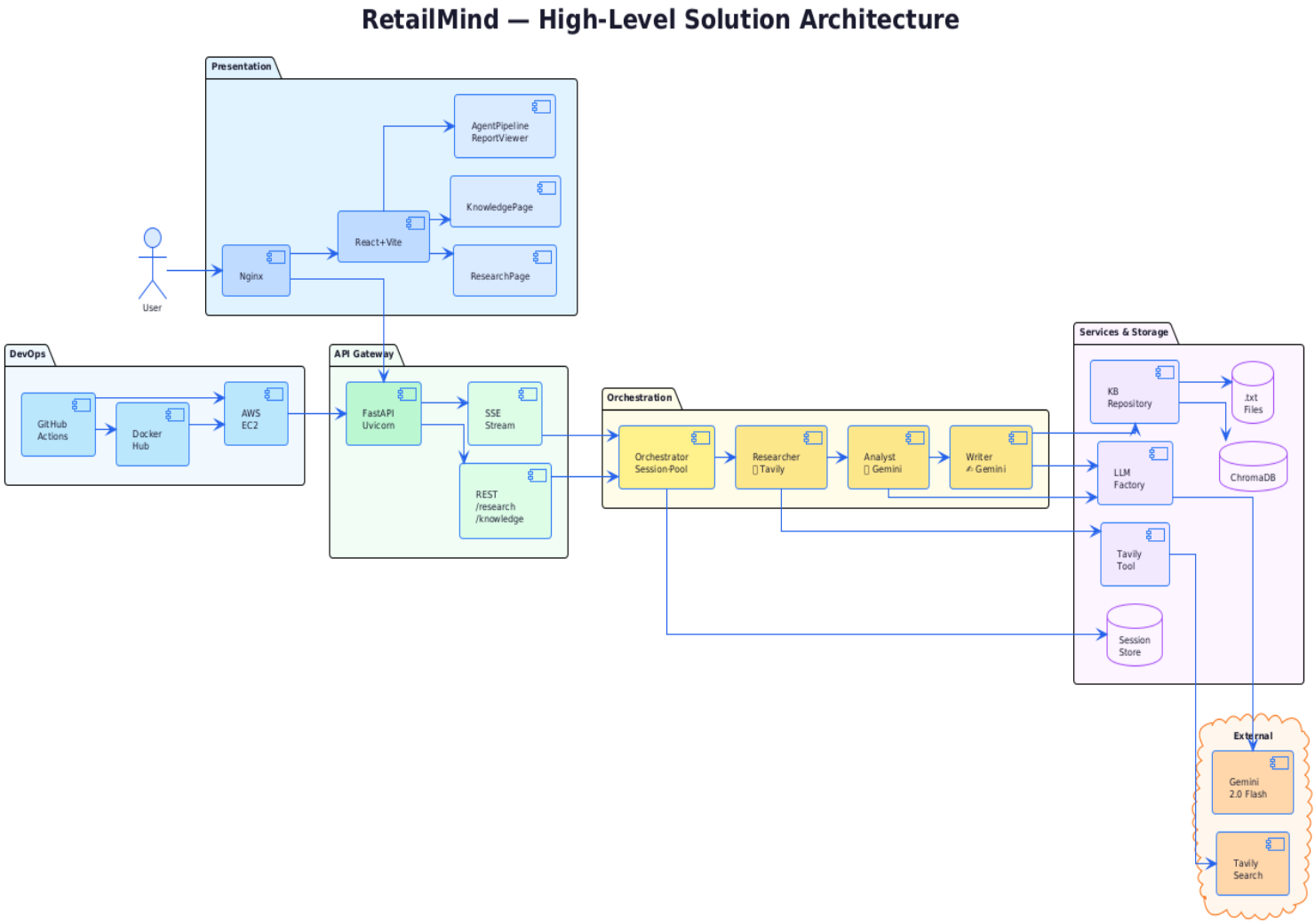

Figure 4.1: High-Level RetailMind Solution Architecture

4.2 Three-Agent AI Architecture

All three agents share the Google Gemini 2.0 Flash LLM instance from the get_llm_for_crewai() factory. The TavilySearchTool wraps TavilyClient with a CrewAI-compatible run() interface.

Agent 1 — Senior Retail Industry Researcher: Goal: gather the most relevant, up-to-date retail information via multiple search angles. Tools: TavilySearchTool. max_iter: 6.
Agent 2 — Retail Market Intelligence Analyst: Goal: extract key insights, trends, statistics, and company examples from the Researcher's findings. max_iter: 4.
Agent 3 — Retail Research Report Writer: Goal: produce a comprehensive, professionally structured markdown report. Tools: None. max_iter: 3.

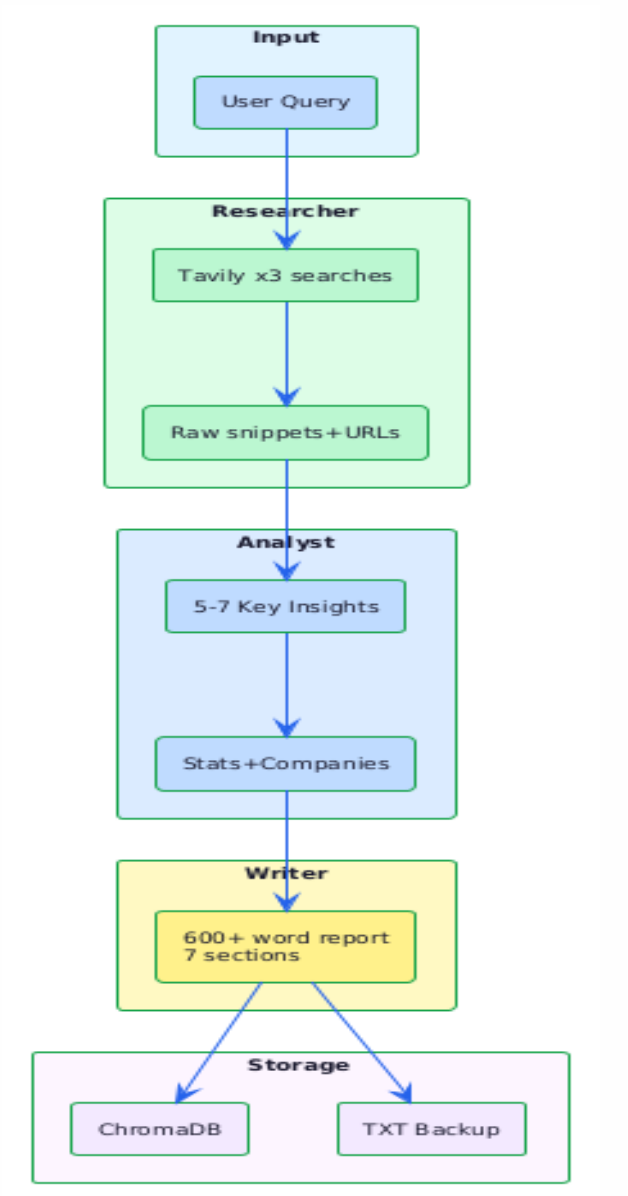

Figure 4.2: Three-Agent Sequential Execution and Data Flow

4.3 Backend Architecture

main.py: FastAPI app factory with CORS middleware and lifespan context manager for startup validation.
api/routes.py: All 8 REST and SSE endpoints.
agents/crew_agents.py: build_agents() — constructs the three CrewAI Agent objects.
agents/crew_tasks.py: build_tasks() — constructs three Task objects with descriptions and agent bindings.
agents/orchestrator.py: Session management, async pipeline execution (ThreadPoolExecutor), SSE generator.
core/config.py: Centralised Settings with startup validation. Raises RuntimeError with actionable message if keys are missing.
core/llm.py & core/search.py: Gemini LLM factory and TavilySearchTool.
knowledge_base/repository.py: KnowledgeRepository — dual-write ChromaDB and plain-text with save_report(), semantic_search(), keyword_search().

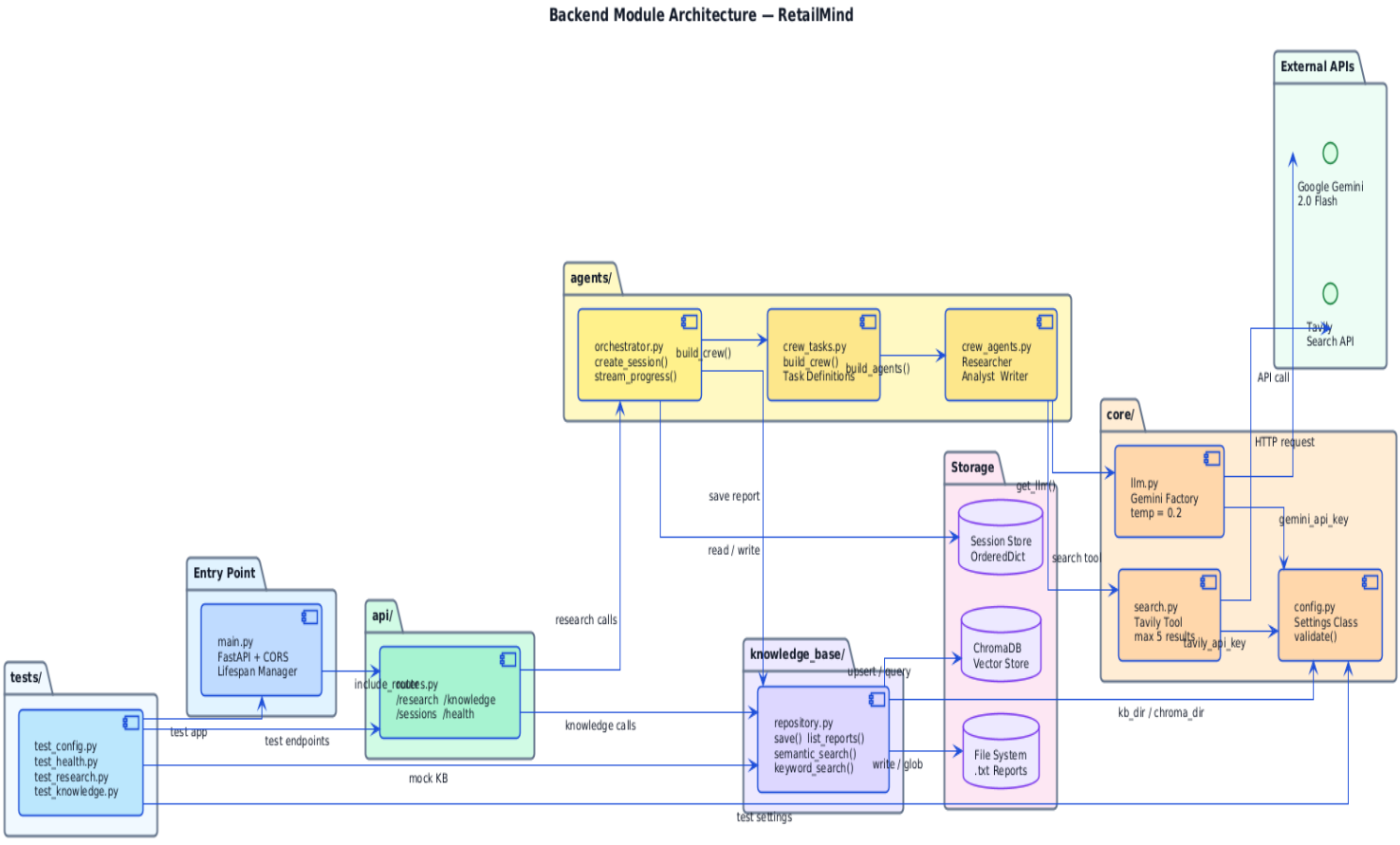

Figure 4.3: Backend Module Architecture

4.4 Frontend Architecture

App.jsx: React Router routes for /, /knowledge, /history.
components/AgentPipeline.jsx: Live pipeline visualisation with per-agent status indicators consuming SSE events.
components/ReportViewer.jsx: Markdown renderer for completed reports.
pages/ResearchPage.jsx: Query input form + live pipeline + report view.
pages/KnowledgePage.jsx: Semantic and keyword search over the knowledge base.
pages/HistoryPage.jsx: Past session browser.

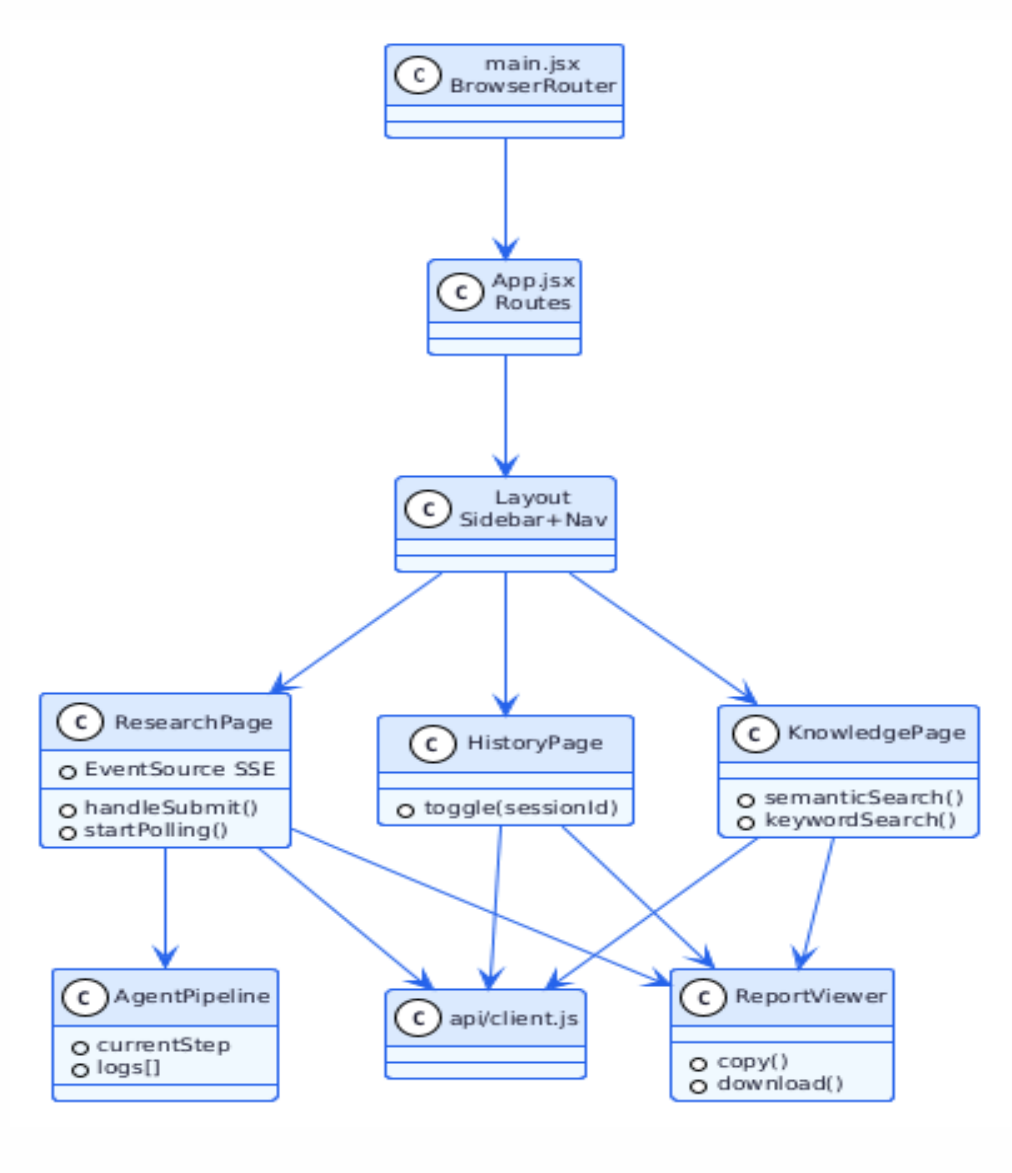

Figure 4.4: React Frontend Component Hierarchy

4.5 DevOps Architecture

4.5.1 Containerisation (Docker)

retailmind-backend: python:3.11-slim, port 8000, mounts kb_data and chroma_data volumes, health check on /api/health.
retailmind-frontend: Node 20 Alpine build stage → nginx:alpine runtime, port 80, proxies /api/* to backend.

4.5.2 CI/CD Automation (GitHub Actions)

ci.yml: Runs on every push and PR. Jobs: backend-tests (ruff lint + pytest), frontend-tests (npm ci + build), docker-build-check.
deploy.yml: Runs on push to main. Jobs: test → build (push :latest + :<sha> to Docker Hub) → deploy (SSH to EC2, compose up, log SHA).
rollback.yml: Manually triggered. Reads deployments.log for previous SHA, pulls that image, restarts containers.

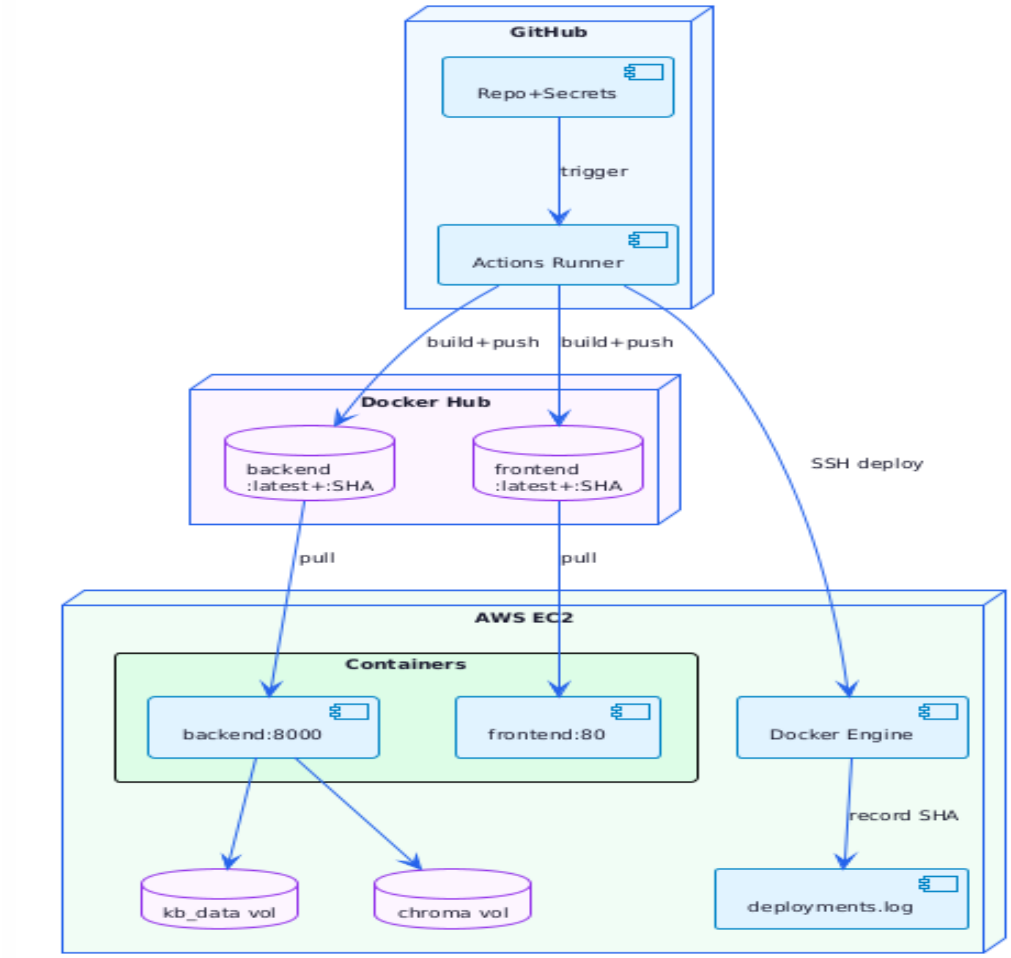

Figure 4.5: DevOps Architecture

CHAPTER 5
DESIGNS

5.1 Algorithm

5.1.1 Three-Agent Research Algorithm

```
Algorithm: RetailMind_Research_Pipeline
Input:  query (natural language string)
Output: research_report (markdown report)

1. VALIDATE: Check GEMINI_API_KEY and TAVILY_API_KEY.
2. CREATE SESSION: session_id = UUID(); state = {status:pending, step:0, report:None}
3. ASYNC EXECUTE (ThreadPoolExecutor):
   a. BUILD: researcher, analyst, writer = build_agents()
   b. BUILD: t1, t2, t3 = build_tasks(query)
   c. CREW = Crew([researcher,analyst,writer], [t1,t2,t3], process=sequential)
   d. UPDATE: status=running, step=1
   e. RESEARCHER (max_iter=6): search multiple query variants -> collect snippets
   f. UPDATE: step=2
   g. ANALYST (max_iter=4): synthesise snippets -> structured findings
   h. UPDATE: step=3
   i. WRITER (max_iter=3): produce markdown report from findings
   j. PERSIST: chromadb.add(report) + write_txt(kb_dir/session_id.txt)
   k. UPDATE: status=done, report=research_report
4. SSE STREAM (parallel): poll session state; yield events on change;
   yield final report event when status=done
5. RETURN session_id immediately (async)
```

5.1.2 Rollback Algorithm

```
Algorithm: RetailMind_Rollback
Input:  ROLLBACK_SHA (optional)
Output: running containers at target SHA version

1. SSH into EC2.
2. If ROLLBACK_SHA provided: target = ROLLBACK_SHA
   Else: target = second-to-last line of deployments.log
3. Set IMAGE_TAG = target
4. docker compose pull   (fetches :target images from Docker Hub)
5. docker compose up -d  (restarts with target version)
6. Append rollback record to deployments.log
```

5.2 Function-Oriented Design

Function	Module	Input	Output
build_agents()	agents/crew_agents.py	None	(Researcher, Analyst, Writer) tuple

build_tasks(query)	agents/crew_tasks.py	query: str	(Task1, Task2, Task3) tuple
create_session(query)	agents/orchestrator.py	query: str	session_id: UUID str
semantic_search(query, n)	knowledge_base/repository.py	query: str, n: int	List[dict]
keyword_search(keyword)	knowledge_base/repository.py	keyword: str	List[dict]
get_llm_for_crewai()	core/llm.py	None	ChatGoogleGenerativeAI
start_research(body, bg)	api/routes.py	ResearchRequest	ResearchStartResponse (202)
stream_research(session_id)	api/routes.py	session_id: str	StreamingResponse (SSE)

Table 5.2: Function-Oriented Design

5.3 System Design

5.3.1 Data Flow Diagram

Level 0 — Context Diagram: RetailMind as a single process with two external entities (User and External APIs) producing Research Report and SSE Stream outputs.

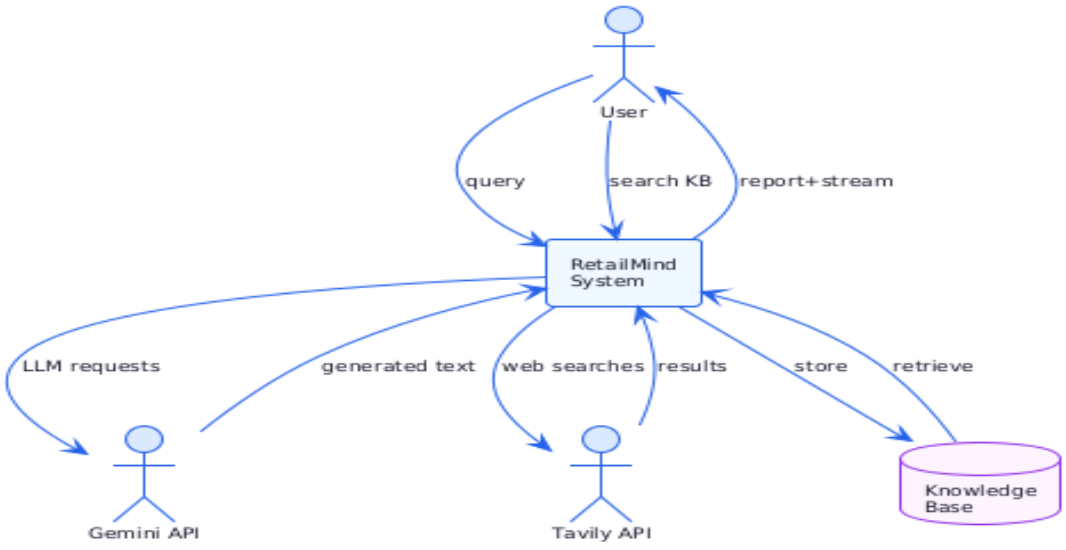

Figure 5.1: DFD Level 0 — Context Diagram

Level 1 — Decomposed DFD: Five internal processes — Session Manager, Research Pipeline, SSE Stream Manager, Knowledge Repository, and Knowledge Search.

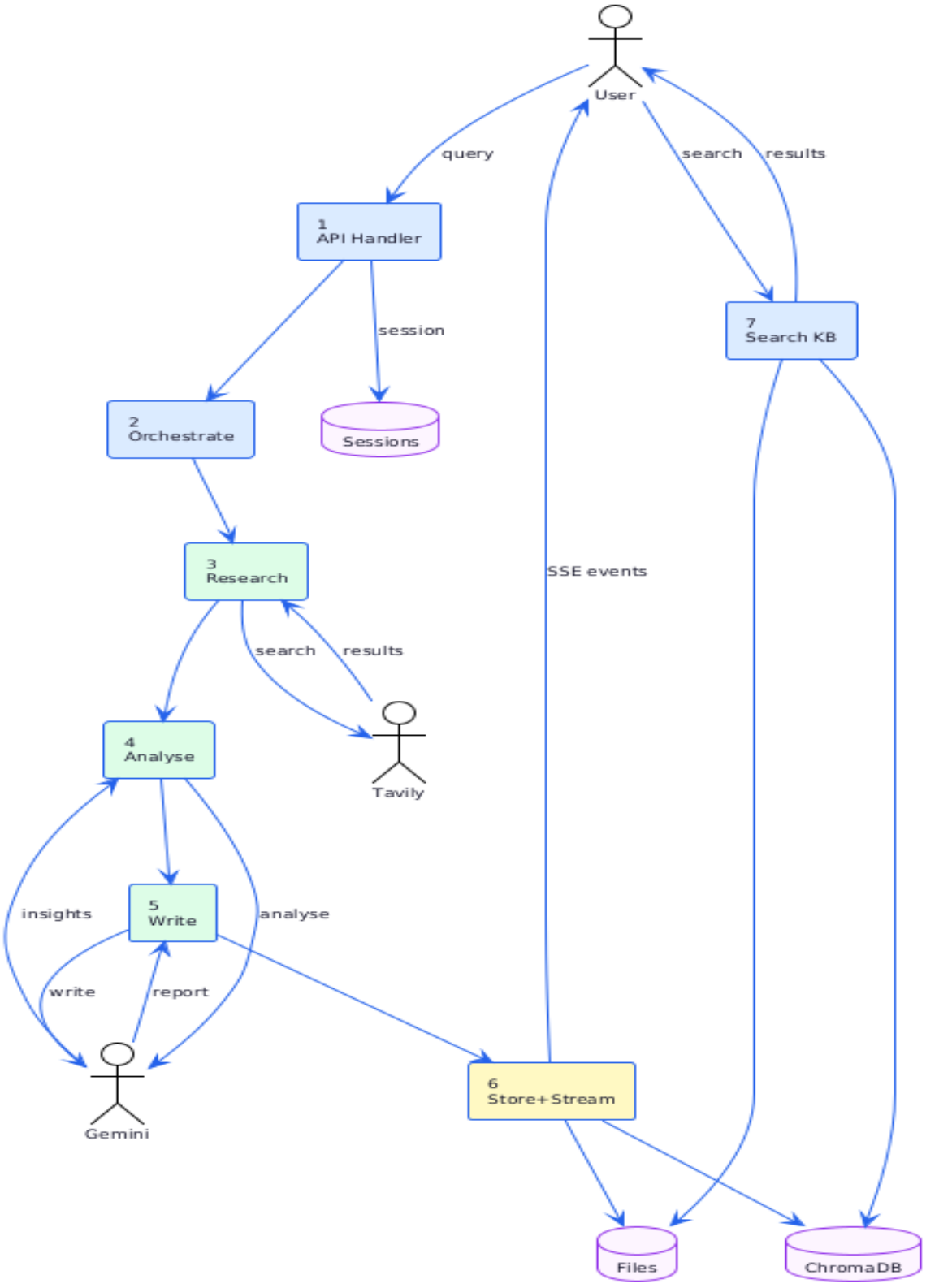

Figure 5.2: DFD Level 1 — Internal Processes

5.3.2 Flowchart

The research request flowchart shows the complete lifecycle from user query submission through asynchronous agent execution to report delivery via SSE.

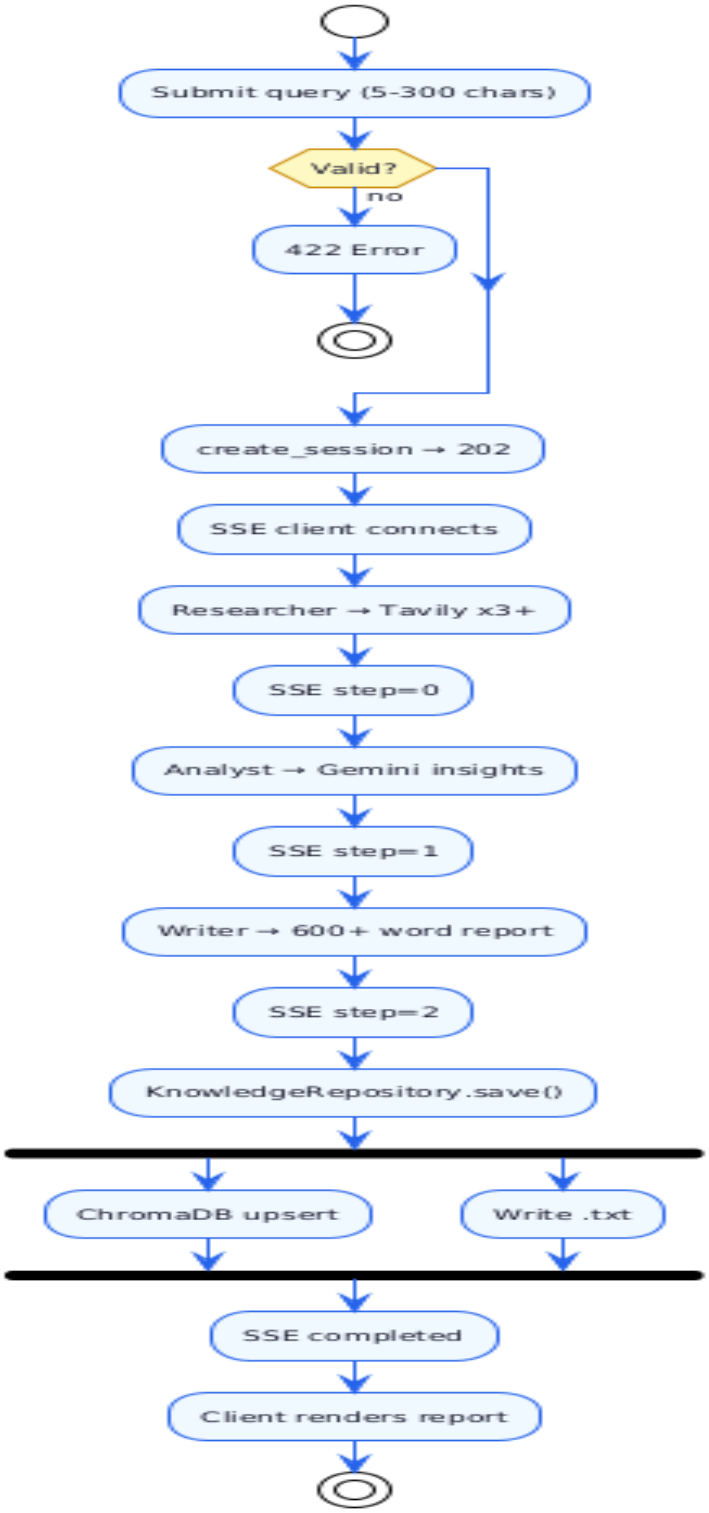

Figure 5.3: Research Pipeline Flowchart

The CI/CD pipeline flowchart shows automation from git push through test, build, push, SSH deploy, and health check stages.

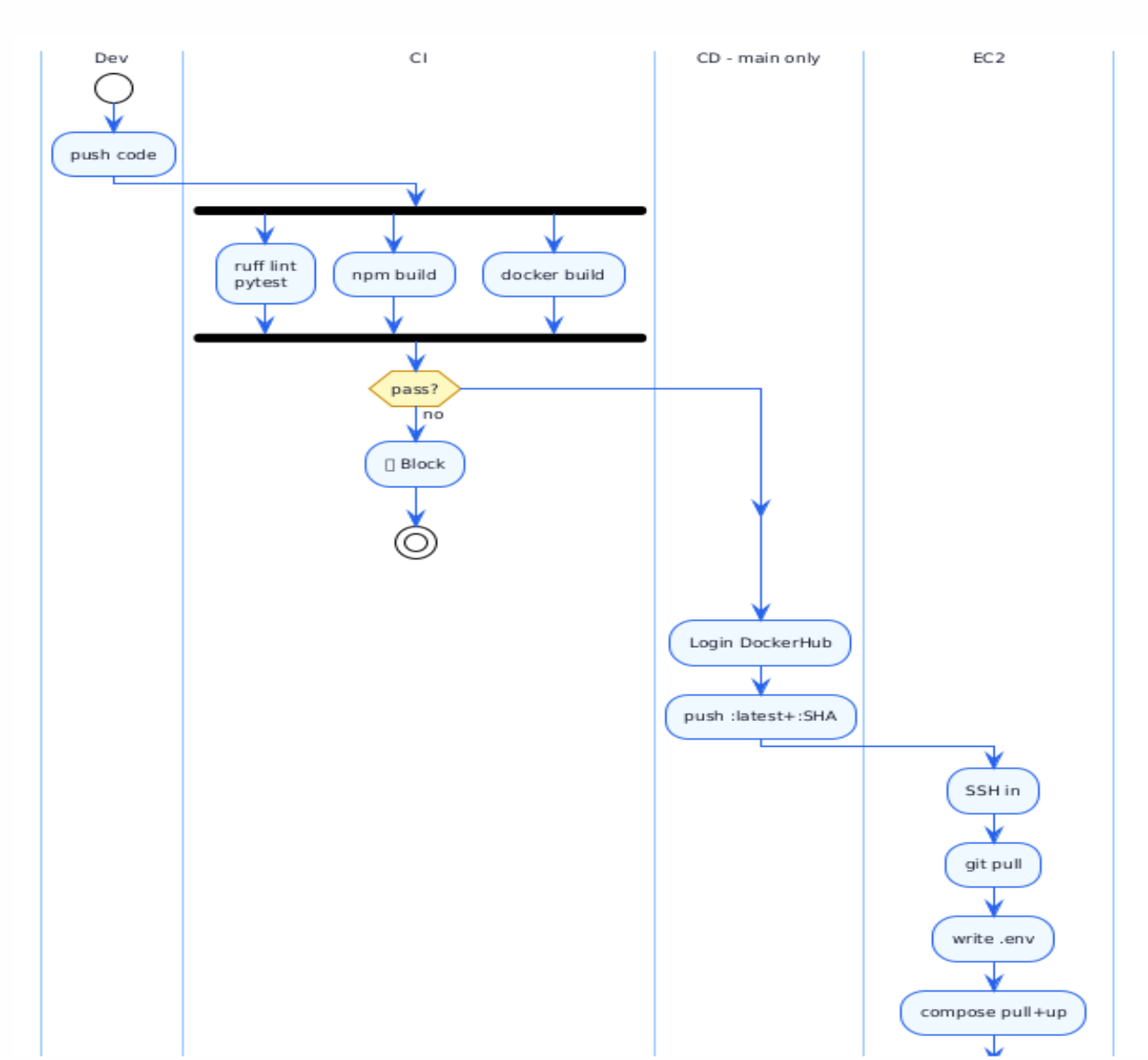

Figure 5.4: CI/CD Pipeline Flowchart

5.3.3 Activity Diagrams

The research session activity diagram shows parallel swimlanes for Frontend (EventSource), Backend API (SSE generator), Agent Pipeline (ThreadPoolExecutor), and Knowledge Base.

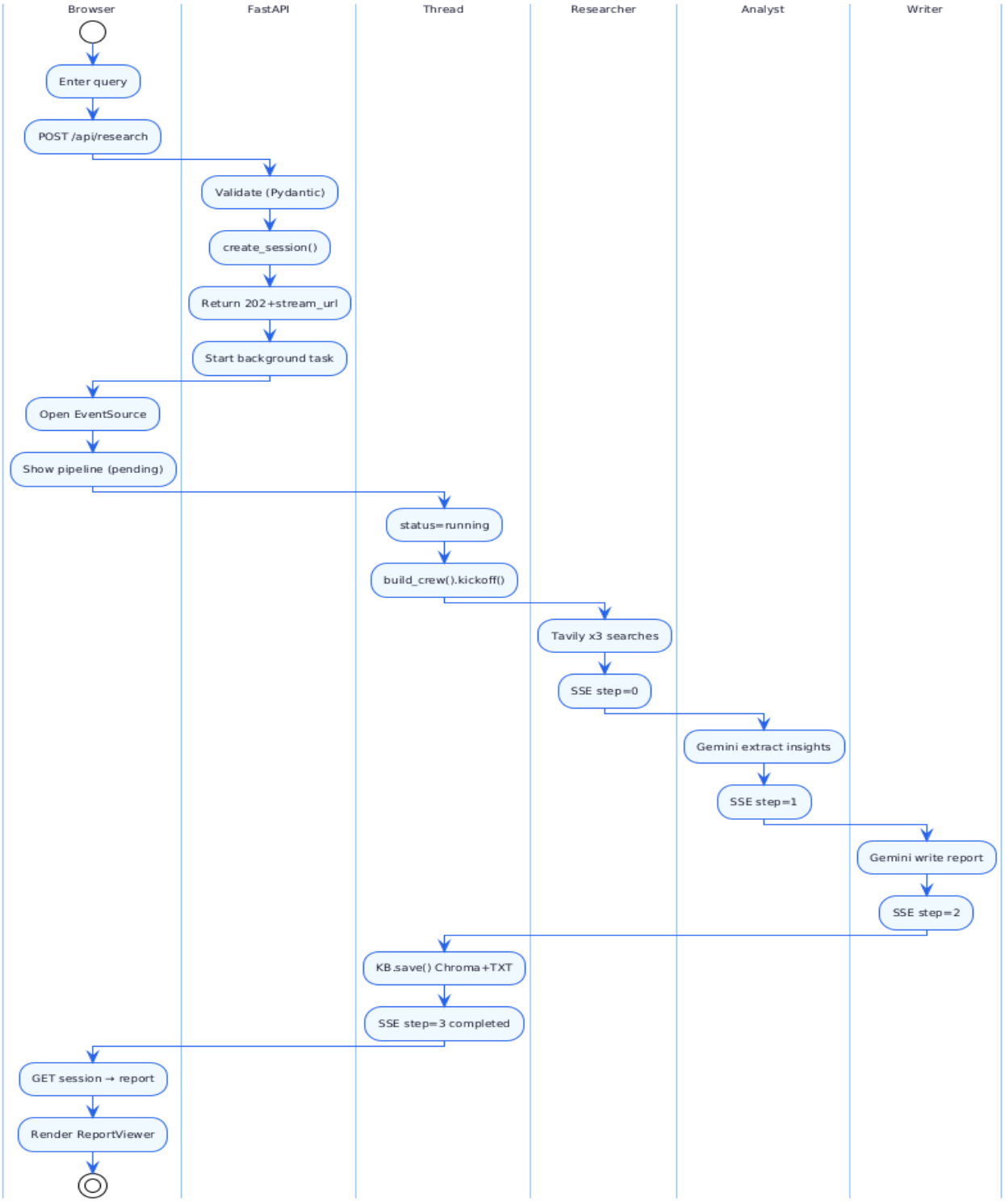

Figure 5.5: Activity Diagram — Research Session Lifecycle

5.3.4 Class Diagram Overview

Key backend classes and their relationships:

Settings (core/config.py): Reads all configuration from environment variables. Method: validate().
TavilySearchTool (core/search.py): CrewAI-compatible tool wrapping TavilyClient. Method: run(query).
KnowledgeRepository (knowledge_base/repository.py): Manages ChromaDB + file store. Methods: save_report(), semantic_search(), keyword_search(), stats().
ResearchRequest / ResearchStartResponse (Pydantic Models): Request validation and response schema.

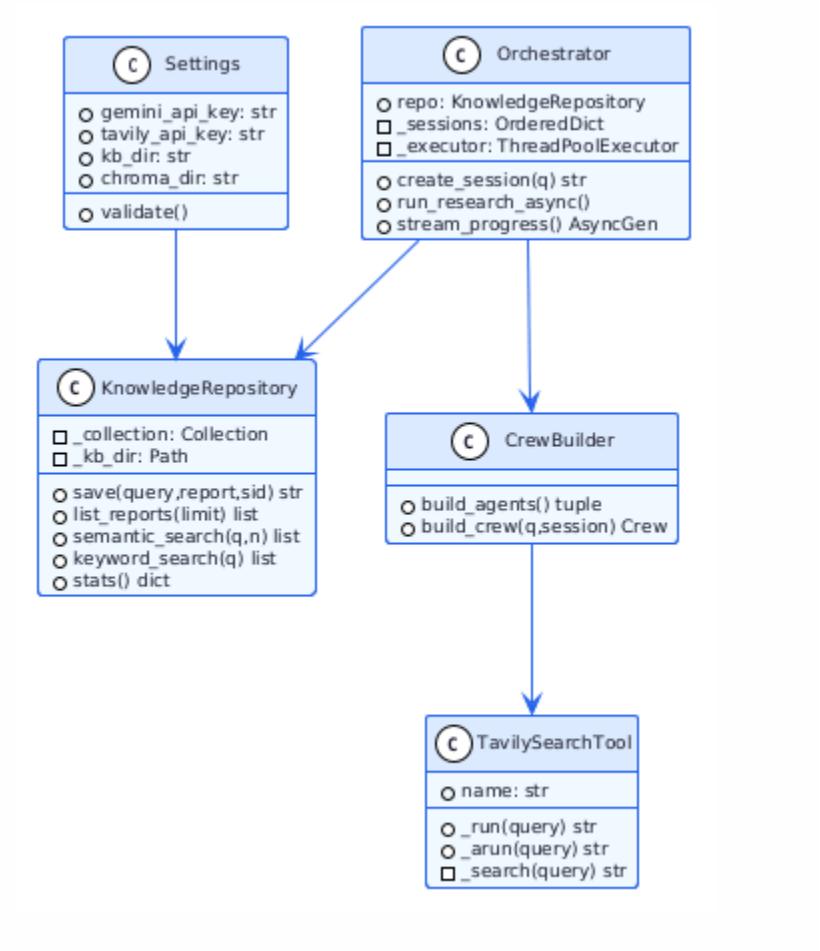

Figure 5.6: Class Diagram — Backend Core Classes

5.4 Database Design

RetailMind uses a single ChromaDB collection named retail_research:

Field	Type	Description
id	string (UUID)	Session ID — unique identifier for each report
document	string	Full markdown text of the generated research report
embedding	float[384]	Dense vector from ONNX all-MiniLM-L6-v2 model
metadata.query	string	Original research query
metadata.session_id	string	Session ID (for metadata filtering)
metadata.timestamp	ISO 8601 string	Datetime when the report was generated
metadata.file_path	string	Path to the plain-text backup file

Table 5.4: ChromaDB Collection Schema

Plain-text backup files follow this structure: metadata header (Query, Session, Timestamp) followed by the full markdown report body.

CHAPTER 6
IMPLEMENTATION, TESTING AND MAINTENANCE

6.1 Key Implementation Details

6.1.1 CrewAI Pipeline

```
# agents/orchestrator.py
def _run_crew(session_id: str, query: str):
    researcher, analyst, writer = build_agents()
    t1, t2, t3 = build_tasks(query, researcher, analyst, writer)
    crew = Crew(agents=[researcher, analyst, writer],
                tasks=[t1, t2, t3], process=Process.sequential)
    return str(crew.kickoff(inputs={'query': query}))
```

6.1.2 SSE Streaming

```
async def stream_progress(session_id: str):
    last_step = -1
    while True:
        s = _sessions[session_id]
        if s['current_step'] != last_step:
            last_step = s['current_step']
            yield f"data: {json.dumps({'step': last_step})}\n\n"
        if s['status'] in ('done', 'error'):
            yield f"data: {json.dumps({'done': True, 'report': s['report']})}\n\n"
            break
        await asyncio.sleep(0.5)
```

6.1.3 Frontend Dockerfile (Multi-Stage)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

6.2 Testing Strategy

6.2.1 Unit Testing

Test File	What it Tests
test_config.py	Settings reads env vars; validate() raises with clear message when keys missing

test_health.py	GET /api/health returns 200 {status: ok}
test_research.py	POST /api/research returns 202 with session_id; SSE endpoint returns correct content-type
test_knowledge.py	Knowledge endpoints return correct schema; semantic search handles empty query

6.2.2 Integration and System Testing

Full research pipeline: report generated within 3 minutes, all agents complete, report saved to both stores.

Knowledge base persistence: reports remain accessible after container restart.

CI/CD pipeline: all three jobs complete, new version live on EC2 within 10 minutes of a push to main.

Rollback: previous version running within 2 minutes of triggering the rollback workflow.

6.3 Installation Instructions

6.3.1 Local Setup (Docker Compose)

```
git clone https://github.com/awwniket47/Retailmind-Autonomous_Retail_Research_Agent.git
cd Retailmind-Autonomous_Retail_Research_Agent
cp backend/.env.example backend/.env
# Add GEMINI_API_KEY and TAVILY_API_KEY to backend/.env
docker compose up --build
# Frontend: http://localhost:80    API Docs: http://localhost:8000/docs
```

Variable	Required	Description
GEMINI_API_KEY	Yes	Google AI Studio API key
TAVILY_API_KEY	Yes	Tavily Search API key
KB_DIR	No (./kb_data)	Plain-text report backup directory
CHROMA_DIR	No (./chroma_db)	ChromaDB persistent vector store directory

Table 6.3: Environment Variables

6.3.2 AWS EC2 Production Deployment

```
# On EC2: install Docker, configure GitHub Secrets, then push to main.
# GitHub Actions will automatically test, build, push, and deploy.
# To rollback: GitHub Actions -> Rollback workflow -> Run workflow
```

CHAPTER 7
RESULTS AND DISCUSSIONS

7.1 Application Screenshots

7.1.1 Research Interface

Users enter a natural language query and click Research. The interface validates input (5–300 characters) and transitions to the live pipeline view.

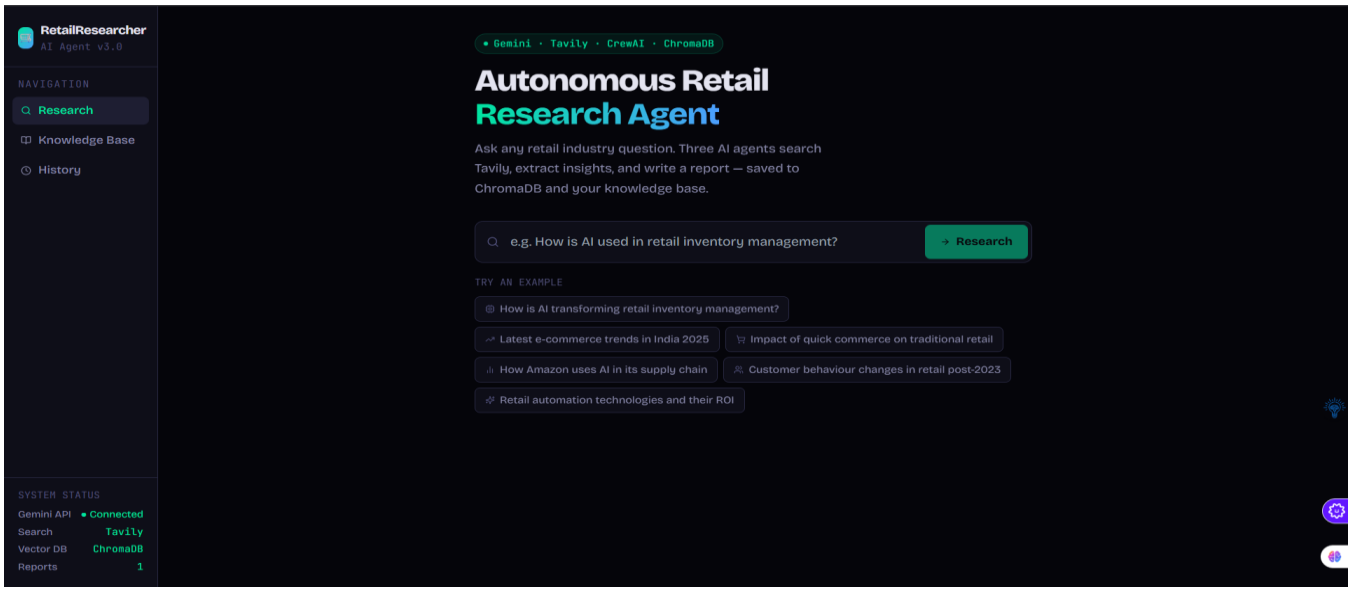

Figure 7.1: RetailMind Research Page — Query Input Interface

7.1.2 Live Agent Pipeline Streaming

As the pipeline executes, the frontend receives SSE events and updates the AgentPipeline component in real time — showing active agent (pulsing), completed agents (checkmark), and live log lines.

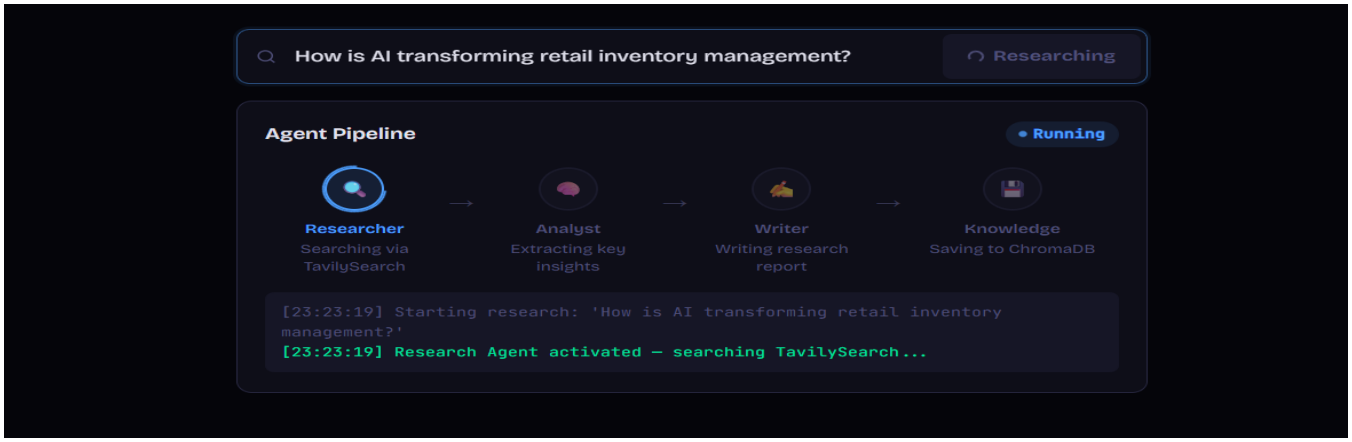

Figure 7.2: Live Agent Pipeline Streaming View

7.1.3 Generated Research Report

On Writer completion, the full report is rendered in formatted markdown with heading hierarchy, bullet points, statistics, and source references. A typical report includes Executive Summary, Key Trends, Company Examples, and Actionable Insights.

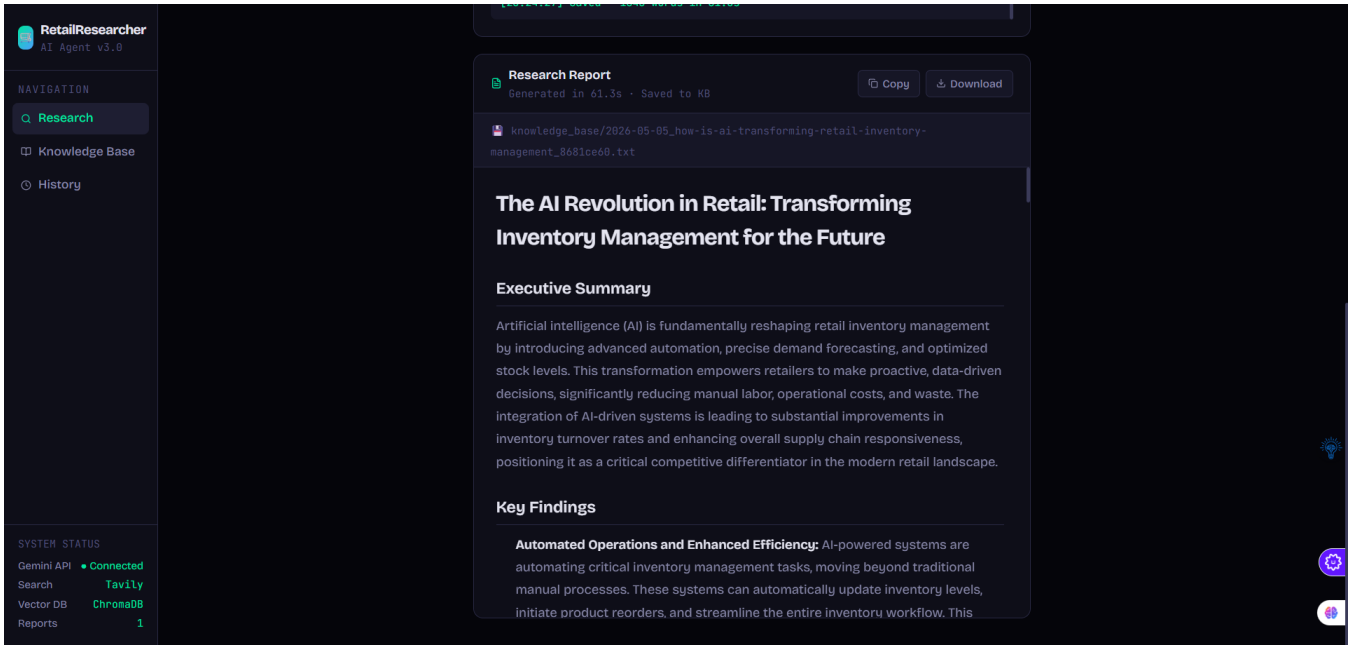

Figure 7.3: Generated Research Report View

7.1.4 Knowledge Base Search

The Knowledge Base page provides semantic similarity search (ChromaDB) and keyword search over all previously generated reports

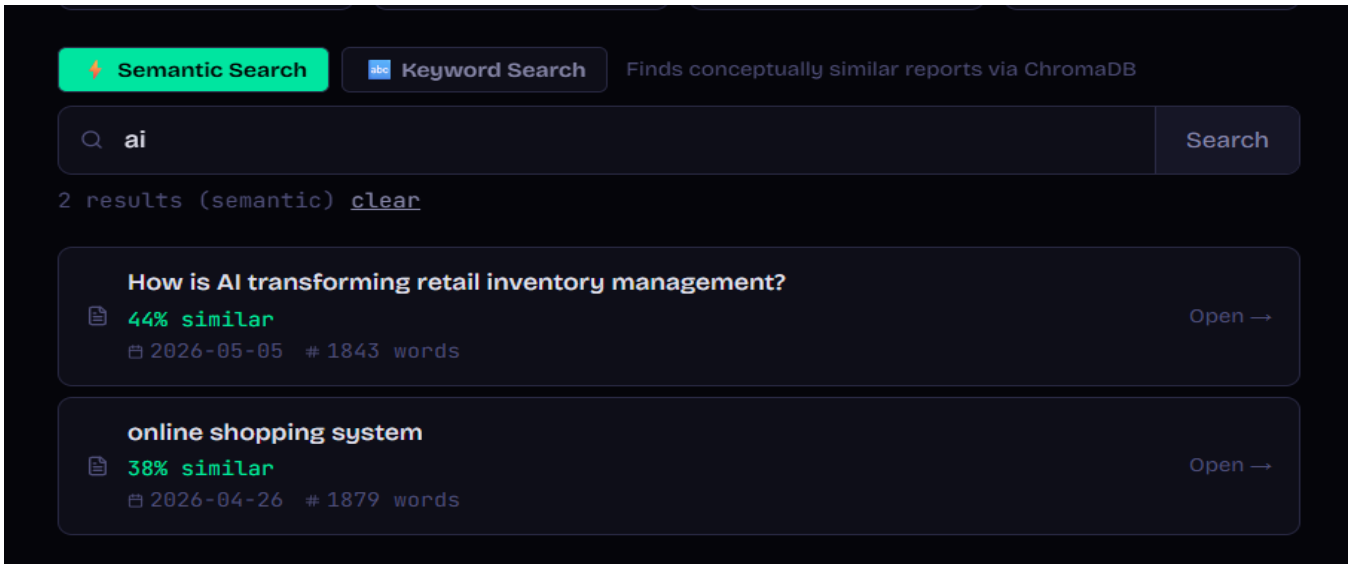

Figure 7.4: Knowledge Base Semantic Search Interface

7.2 DevOps Infrastructure Results

7.2.1 GitHub Actions CI and CD Pipelines

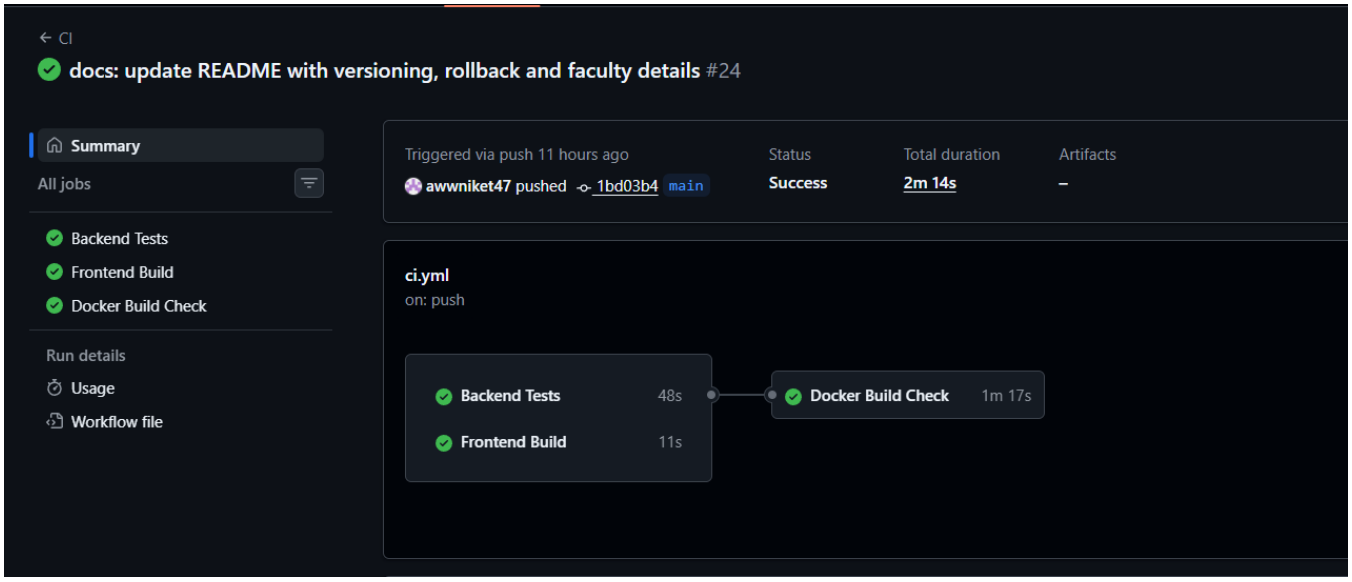

Figure 7.5: GitHub Actions CI Run — All Tests Passing

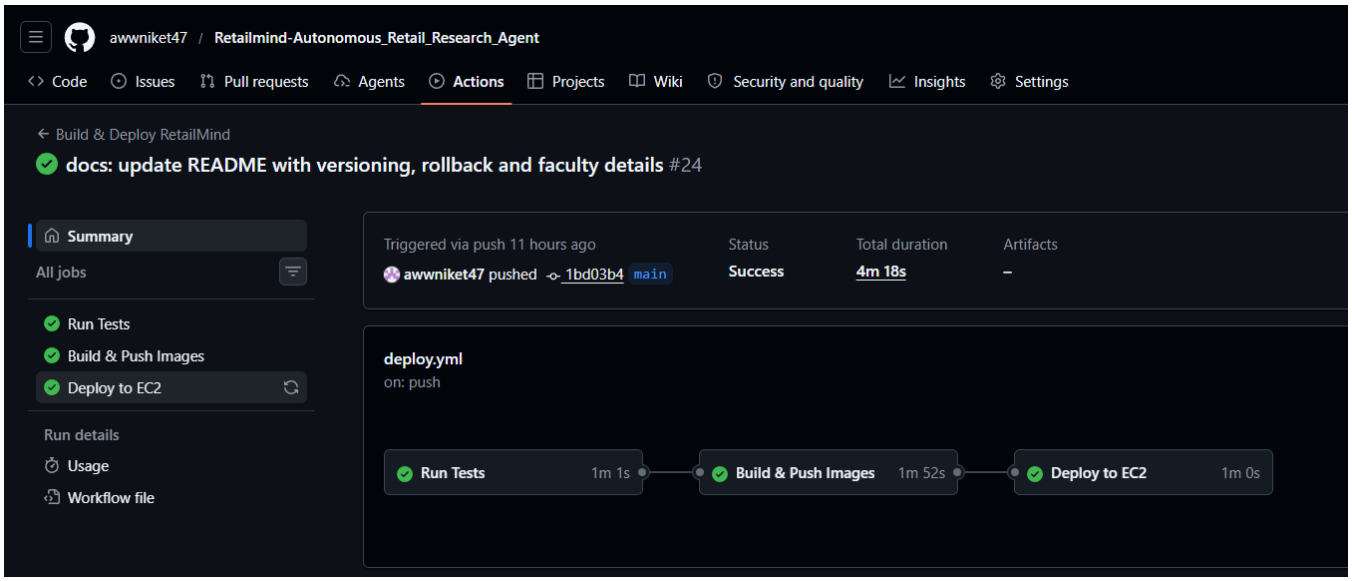

Figure 7.6: GitHub Actions CD Run — Test, Build, Deploy Completed

7.2.2 Docker Hub and AWS EC2

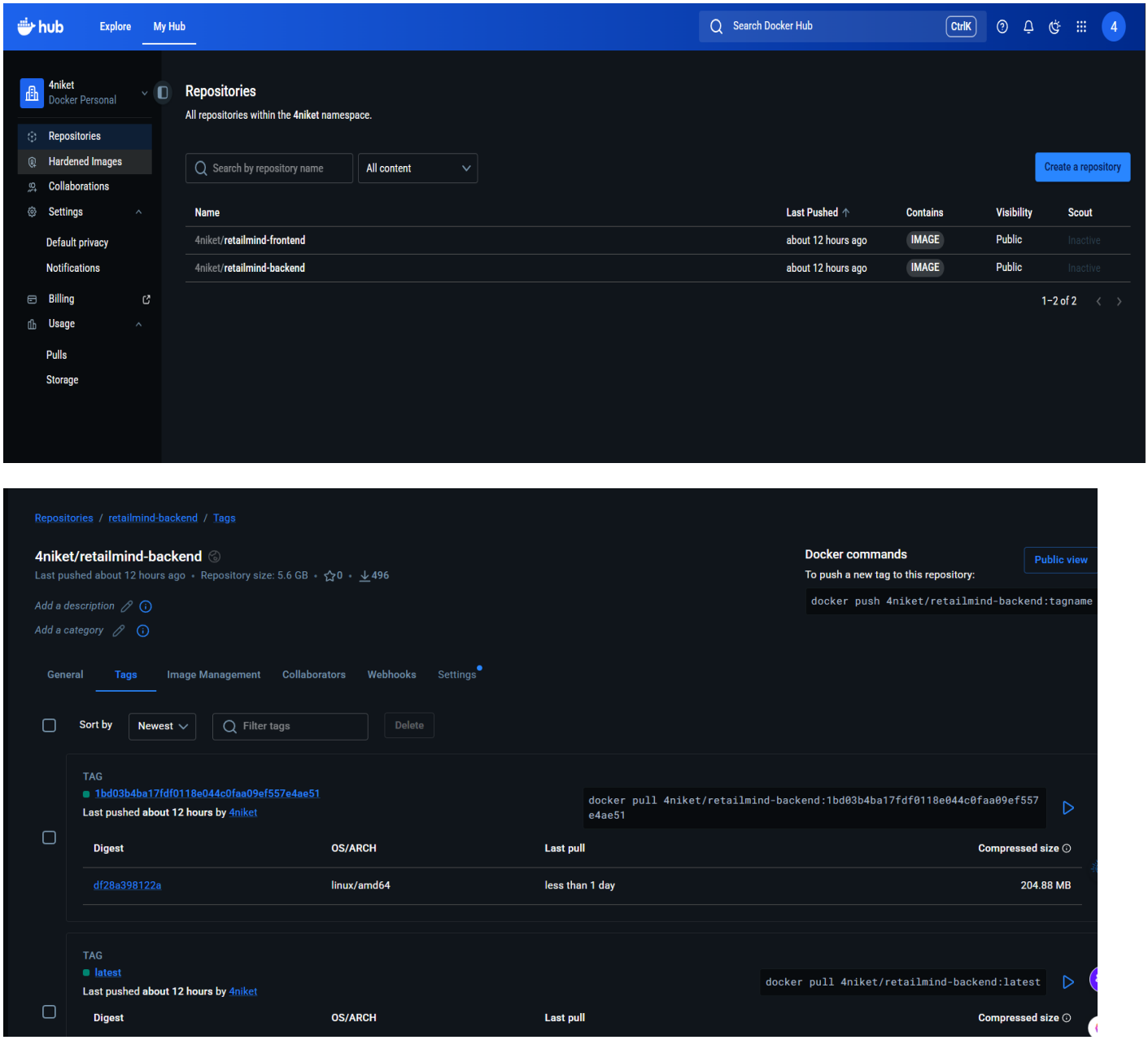

Figure 7.7: Docker Hub — SHA-Tagged Image Versions

Figure 7.8: AWS EC2 — docker compose ps Showing Both Containers Healthy

7.3 Discussions

The three-agent specialisation produced measurably better reports than early single-agent prototypes. Separating search, synthesis, and writing — each with a tailored prompt and backstory — prevented scope creep and reduced hallucination.

SSE streaming transformed a multi-minute wait into a transparent, observable workflow. The SHA-versioned deployment strategy proved its value during testing when a breaking change was immediately rolled back in under two minutes.

The dual-storage knowledge base (ChromaDB + plain-text) provided both powerful semantic retrieval and a resilient human-readable archive. This redundancy is recommended for any AI application that accumulates generated content over time.

CHAPTER 8
SUMMARY AND CONCLUSIONS

8.1 Summary

This report has documented the complete design, development, and deployment of RetailMind — an autonomous retail research platform built across three courses: GenAI (three-agent CrewAI pipeline), Agentic AI (SSE streaming and ChromaDB knowledge base), and DevOps (Docker, GitHub Actions CI/CD, AWS EC2).

The system accepts a natural language query, autonomously executes a Researcher → Analyst → Writer pipeline powered by Google Gemini 2.0 Flash and Tavily Search, streams live progress to the browser, persists generated reports in dual storage, and deploys via a fully automated, SHA-versioned pipeline with one-click rollback.

8.2 Conclusions

Multi-agent role specialisation is more effective than single-agent approaches for complex multi-stage cognitive tasks. Production DevOps practices — automated testing, SHA-versioned deployments, health-check-driven startup, and rollback — are essential, not optional, for AI applications. SSE streaming significantly enhances user experience by making AI processing transparent.

Docker Compose on a single EC2 instance is a pragmatic and cost-effective production deployment strategy for portfolio-scale AI applications, providing all containerisation benefits without Kubernetes overhead.

CHAPTER 9
FUTURE SCOPE

9.1 AI Pipeline Enhancements

User-configurable research depth (Quick / Standard / Deep) controlling Researcher max_iter.
Industry vertical specialisation — domain-specific agent configs for fashion, grocery, electronics, luxury.
Automatic source citation with clickable URLs embedded in generated reports.
Parallel Researcher sub-agents exploring different topic angles simultaneously.
Custom report templates (Competitive Analysis, Market Entry, Trend Report).

9.2 Application Features

User authentication with per-user ChromaDB namespaces for private research libraries.
PDF and Word export of generated reports using reportlab or python-docx.
Scheduled recurring research tasks with email notifications on significant changes.
Real-time market data integration (Yahoo Finance, Alpha Vantage) for live financial metrics.

9.3 DevOps and Infrastructure

Kubernetes (K3s) migration for horizontal pod autoscaling and rolling deployments.
Terraform IaC for reproducible AWS environment provisioning from code.
Prometheus + Grafana monitoring stack for production observability.
Trivy container image vulnerability scanning in the CI pipeline.

CHAPTER 10
APPENDIX

10.1 API Endpoint Reference

Endpoint	Method	Description
/api/health	GET	Health check — returns {status: ok}
/api/research	POST	Start new research session — returns session_id and stream_url
/api/research/{id}/stream	GET	SSE stream for live agent progress
/api/research/{id}	GET	Get session state and completed report
/api/sessions	GET	List all research sessions
/api/knowledge	GET	List all saved reports
/api/knowledge/stats	GET	Knowledge base statistics
/api/knowledge/search/semantic	GET	Natural language similarity search
/api/knowledge/search/keyword	GET	Keyword text search

10.2 Project File Structure

```
Retailmind-Autonomous_Retail_Research_Agent/
+-- backend/
|   +-- Dockerfile, requirements.txt, main.py
|   +-- api/routes.py
|   +-- agents/  crew_agents.py, crew_tasks.py, orchestrator.py
|   +-- core/    config.py, llm.py, search.py
|   +-- knowledge_base/repository.py
|   +-- tests/   test_config.py, test_health.py, test_research.py, test_knowledge.py
+-- frontend/
|   +-- Dockerfile, nginx.conf, package.json
|   +-- src/  App.jsx, api/client.js
|         components/  AgentPipeline.jsx, ReportViewer.jsx, Layout.jsx
|         pages/       ResearchPage.jsx, KnowledgePage.jsx, HistoryPage.jsx
+-- .github/workflows/  ci.yml, deploy.yml, rollback.yml
+-- docker-compose.yml
GitHub: https://github.com/awwniket47/Retailmind-Autonomous_Retail_Research_Agent
```

CHAPTER 11
BIBLIOGRAPHY

Research Papers and Books

[1] P. Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' NeurIPS, 2020.

[2] S. Yao et al., 'ReAct: Synergizing Reasoning and Acting in Language Models,' ICLR, 2023.

[3] J. Wei et al., 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,' arXiv:2201.11903, 2022.

[4] A. Vaswani et al., 'Attention Is All You Need,' NeurIPS, 2017.

[5] N. Reimers et al., 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,' EMNLP, 2019.

[6] M. Zaharia et al., 'The Shift from Models to Compound AI Systems,' Berkeley AI Research Blog, 2024.

Technical Documentation and Online Resources

[7] Google AI, Gemini 2.0 Flash API Documentation, 2025. Available: https://ai.google.dev/gemini-api/docs. [Accessed: Apr. 2026].

[8] FastAPI Developers, FastAPI Documentation, 2025. Available: https://fastapi.tiangolo.com. [Accessed: Apr. 2026].

[9] CrewAI, CrewAI Documentation, 2024. Available: https://docs.crewai.com. [Accessed: Apr. 2026].

[10] LangChain, LangChain Python Documentation, 2024. Available: https://python.langchain.com. [Accessed: Apr. 2026].

[11] Tavily, Tavily Search API Documentation, 2024. Available: https://docs.tavily.com. [Accessed: Apr. 2026].

[12] ChromaDB, ChromaDB Documentation, 2024. Available: https://docs.trychroma.com. [Accessed: Apr. 2026].

[13] Docker Inc., Docker Documentation, 2025. Available: https://docs.docker.com. [Accessed: Apr. 2026].

[14] GitHub, GitHub Actions Documentation, 2025. Available: https://docs.github.com/en/actions. [Accessed: Apr. 2026].

[15] Amazon Web Services, AWS EC2 User Guide, 2025. Available: https://docs.aws.amazon.com/ec2. [Accessed: Apr. 2026].

[16] React, React 18 Documentation, Meta, 2024. Available: https://react.dev. [Accessed: Apr. 2026].

[17] MDN Web Docs, Server-Sent Events, Mozilla, 2024. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events. [Accessed: Apr. 2026].

[18] Pydantic Developers, Pydantic V2 Documentation, 2024. Available: https://docs.pydantic.dev/latest. [Accessed: Apr. 2026].

GitHub Repositories

[19] 'RetailMind — Autonomous Retail Research Agent,' GitHub, 2026. Available: https://github.com/awwniket47/Retailmind-Autonomous_Retail_Research_Agent.

[20] CrewAI, 'crewAI — Multi-agent orchestration framework,' GitHub, 2024. Available: https://github.com/crewAIInc/crewAI.

[21] ChromaDB, 'chroma — AI-native open-source embedding database,' GitHub, 2024. Available: https://github.com/chroma-core/chroma.

Retailmind-Autonomous Retail Research Agent